

Production Dashboard API — API Specification Design (API仕様設計)

003_DD-API — supports 003_DD; handler designed in 003_DD-FN; requirements REQ-028–REQ-042.

Document control (改版履歴)

Field	Value
Document ID	003_DD-API
System name	ProductionManagementAI
Subsystem name	Production orders
Work item	WI-004
Created by	Claude (for ThanhTN)
Created date	2026-09-22
Last updated by	Claude (for ThanhTN)
Last updated date	2026-09-22

Version	Date	Author	Revision content
1	2026-09-22	Claude (for ThanhTN)	Initial creation
2	2026-09-22	Claude (for ThanhTN)	<code>GET /api/system/health</code> added (DEC-017, DEC-019, DEC-020); <code>workload[].weekStart</code> clarified for the current week
3	2026-09-23	Claude (for ThanhTN)	WI-005: the UI is Japanese. Example product names are Japanese automobile parts (WI-005 DEC-002); the one quoted UI text gives the Japanese with an English gloss (WI-005 DEC-003). No field, rule, API or processing changes

Overview and operation catalog

Field	Value
API / module name	Production dashboard API
Overview	One read-only JSON endpoint behind SCR-003 that returns every widget's figures from one consistent snapshot (DEC-011, DEC-015). It takes no parameters and writes nothing (REQ-039)

No	Endpoint / method	Handler	Purpose	Notes
1	GET /api/dashboard	DashboardController.Get → DashboardService.GetSnapshotAsync	The dashboard snapshot	New in WI-004
2	GET /api/system/health	SystemController.Health → SystemHealthService.CheckAsync	Database reachability for the health indicator	New in WI-004 (DEC-017)

Common rules, unchanged from 001_DD-API and 002_DD-API:

- **Auth:** cookie session (0002_ADR) plus policy `ProductionOrderEditor` = role `Admin` or `Operator`. No session → 401 with an empty body; session without the role → 403 with an empty body. The existing policy is reused for the same reason 002_DD-API gives: a separate read-only policy would imply a permission split that WI-001 DEC-015 has not decided.
- **Content type:** `application/json`; errors `application/problem+json` (RFC 9457) with the 001_DD-API shape. No stack trace or exception text is ever returned.
- **JSON:** camelCase; dates `YYYY-MM-DD` (plant-local); timestamps ISO 8601 UTC with `Z`; enums as strings.
- **CSRF:** not applicable — `GET`, no state change, no body (WI-002 DEC-020 posture unchanged).
- **Caching:** the response carries `Cache-Control: no-store`. The figures are a point-in-time snapshot, so a browser or proxy must never answer a later visit to `/` from an earlier snapshot.

System-wide API registry entries

ID	Endpoint	Overview	Notes
API-DSH-01	GET /api/dashboard	Production dashboard snapshot	New; first entry of the dashboard group
API-SYS-01	GET /api/system/health	Server and database status for signed-in users	New; the anonymous liveness <code>/health</code> is not an API-registry entry and is unchanged

`API-PRD-01` and `API-PO-01` – `API-PO-04` stay as registered in 001_DD-API and 002_DD-API.

Shared response object: DashboardOrder

One element of `overdue.orders` or `dueSoon.orders`. A deliberately narrow row: the dashboard shows these fields and links nowhere (DEC-006), so it gets neither `isOverdue` (the group says so), `updatedAt`, notes nor a version.

No	Name	Variable name	Type	Length	Required	Repeats (array)	Value mapping	Example	Description	Notes
1	ID	<code>id</code>	string (uuid)	36	yes	no	—	<code>0197e4a0-...</code>	<code>production_orders.id</code>	Used only as a rendering key; never displayed
2	Order number	<code>orderNumber</code>	string	13	yes	no	—	<code>P0-2026-00022</code>		
3	Product	<code>product</code>	object	—	yes	no	Join to <code>products</code>	—	<code>{ "id", "sku", "name" }</code> — the same <code>ProductSummary</code> as <code>002_DD-API</code>	Rendered "{sku} — {name}" (<code>003_BD M-12</code>)
4	Quantity	<code>quantity</code>	integer	—	yes	no	—	<code>2390</code>		
5	Due date	<code>dueDate</code>	string (date)	10	yes	no	—	<code>2026-09-12</code>	Plant-local	<code>003_BD M-15</code>
6	Status	<code>status</code>	string	—	yes	no	Enum name	<code>InProgress</code>	Always <code>Draft</code> or <code>InProgress</code> here	Label client-side (<code>M-11</code>)

Shared response object: OrderGroup

No	Name	Variable name	Type	Required	Repeats (array)	Description
1	Total	<code>total</code>	integer	yes	no	Every order in the group, not only those returned
2	Orders	<code>orders</code>	<code>DashboardOrder[]</code>	yes	yes	At most 10, by due date then order number (<code>D-01</code> , <code>D-02</code>)

Shared response object: CountAndQuantity

No	Name	Variable name	Type	Required	Description
1	Order count	<code>orderCount</code>	integer	yes	0 when none
2	Quantity	<code>quantity</code>	integer (int64)	yes	Sum of <code>quantity</code> ; 0 when none. 64-bit: a sum of 32-bit quantities can exceed 2^{31} , and stays exactly representable in a JavaScript number up to 2^{53}

Endpoint / method design

1. GET /api/dashboard

Field	Value
Description	Return every figure on SCR-003 computed from one consistent snapshot, with the snapshot time and the plant-local date the figures were computed for
Return type	DashboardResponse
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments: none. The endpoint binds no route, query or body parameter; a query string sent with the request is ignored and cannot change any window or size (003_BD §5). There is therefore no 400 response.

Processing overview: authorize, then delegate to `DashboardService.GetSnapshotAsync` (003_DD-FN §1). It reads the clock once, derives every window from the plant-local date (003_DD-FN §2), runs 003_DB's seven statements in one read-only `REPEATABLE READ` transaction (§3) and shapes the result (§4). The response is complete or it is an error — never partial (REQ-028).

Processing flow

Step	Description	Calls
1	Authorize	policy <code>ProductionOrderEditor</code>
2	Build the snapshot	<code>DashboardService.GetSnapshotAsync</code> → 003_DD-FN §1
3	200 with <code>DashboardResponse</code> and <code>Cache-Control: no-store</code>	—

Return value

Type	Name	Description
<code>DashboardResponse</code>	body	See Response fields

Request fields: not applicable — the endpoint takes no input.

Response fields — `DashboardResponse`

No	Name	Variable name	Type	Length	Required	Repeats (array)	Value mapping	Example	Description	Notes
1	Snapshot time	asOf	string (date-time)	—	yes	no	UTC	2026-09-22T05:05:12Z	The instant the clock was read	Shown in plant time (M-17)
2	Plant today	today	string (date)	10	yes	no	T	2026-09-22	The date every figure was computed for	
3	Plant time zone	timeZone	string	—	yes	no	PlantOptions.TimeZone	Asia/Tokyo	Lets the client render asOf in the same zone the figures use, rather than the browser's	
4	Status counts	statusCounts	object	—	yes	no	Q1	—	{ total, draft, inProgress, completed, cancelled }, integers, a status with no orders is 0	REQ-029; total = sum of the four
5	Overdue	overdue	OrderGroup	—	yes	no	Q3a	—	D-01	REQ-030
6	Due soon	dueSoon	OrderGroup	—	yes	no	Q3b	—	D-02	REQ-030
7	Workload	workload	object[]	—	yes	yes (exactly 10)	Q2, zero-filled	—	D-03 buckets in display order	REQ-031, DEC-009
7.1	Bucket kind	workload[].kind	string	—	yes	—	—	week	overdue week later	First is overdue , then 8 × week , last is later

No	Name	Variable name	Type	Length	Required	Repeats (array)	Value mapping	Example	Description	Notes
7.2	Week start	<code>workload[].weekStart</code>	string (date) null	10	yes	—	—	<code>2026-09-28</code>	First date the bucket covers: the Monday of the week, except the current week, whose first date is <code>today</code> (its earlier days are overdue, D-03); <code>null</code> for overdue and later	
7.3	Week end	<code>workload[].weekEnd</code>	string (date) null	10	yes	—	—	<code>2026-10-04</code>	Sunday of the week; <code>null</code> for overdue and later	The table equivalent shows the range (M-16)
7.4	Order count / quantity	<code>workload[].orderCount</code> , <code>.quantity</code>	integer, integer (int64)	—	yes	—	—	<code>7</code> , <code>3150</code>		<code>CountAndQuantity</code> fields
8	Top products	<code>topProducts</code>	object[]	—	yes	yes (0–10)	Q4	—	D-04, open quantity descending, then SKU	REQ-032
8.1	Product	<code>topProducts[].product</code>	<code>ProductSummary</code>	—	yes	—	—	—		
8.2	Open quantity	<code>topProducts[].openQuantity</code>	integer (int64)	—	yes	—	—	<code>2390</code>		
8.3	Active orders	<code>topProducts[].activeOrderCount</code>	integer	—	yes	—	—	<code>2</code>		
9	Completed this week	<code>completedThisWeek</code>	<code>CountAndQuantity</code> + <code>from</code>	—	yes	no	Q5	—	D-05; <code>from</code> = W(T)	REQ-034

No	Name	Variable name	Type	Length	Required	Repeats (array)	Value mapping	Example	Description	Notes
10	Completed this month	completedThisMonth	CountAndQuantity + from	—	yes	no	Q5	—	D-06; from = 1st of T's month	REQ-034
11	On time	onTime	object	—	yes	no	Q5	—	{ onTimeCount, completedCount, windowStart }; windowStart = T - 29. The rate is not sent: the client derives it (M-13), so there is one rounding rule and one place it lives	REQ-035
12	Lead time	leadTime	object	—	yes	no	Q5	—	{ averageDays, orderCount, windowStart }; averageDays is the mean rounded half away from zero to one decimal by the server, or null when orderCount = 0	REQ-037
13	Completion trend	completionTrend	object[]	—	yes	yes (exactly 12)	Q6, zero-filled	—	{ weekStart, weekEnd, orderCount }, oldest first; the last is the current week	REQ-036

The `from` and `windowStart` dates are sent so the table equivalents and the tile captions can state their windows exactly, without the client recomputing the plant calendar.

Example response (abridged):

```

{
  "asOf": "2026-09-22T05:05:12Z",
  "today": "2026-09-22",
  "timeZone": "Asia/Tokyo",
  "statusCounts": { "total": 124, "draft": 35, "InProgress": 25, "completed": 56, "cancelled": 8 },
  "overdue": {
    "total": 15,
    "orders": [
      {
        "id": "0197e4a0-0000-7000-8001-000000000078",
        "orderNumber": "PO-2026-00078",
        "product": { "id": "0197e4a0-0000-7000-8000-000000000107", "sku": "P-1007", "name": "ハブベアリング" },
        "quantity": 470,
        "dueDate": "2026-09-08",
        "status": "InProgress"
      }
    ]
  },
  "dueSoon": { "total": 8, "orders": [] },
  "workload": [
    { "kind": "overdue", "weekStart": null, "weekEnd": null, "orderCount": 15, "quantity": 9612 },
    { "kind": "week", "weekStart": "2026-09-22", "weekEnd": "2026-09-27", "orderCount": 6, "quantity": 3270 },
    { "kind": "later", "weekStart": null, "weekEnd": null, "orderCount": 2, "quantity": 2250 }
  ],
  "topProducts": [
    { "product": { "id": "0197e4a0-0000-7000-8000-0000000001029", "sku": "P-1029", "name": "ドアヒンジ" }, "openQuantity": 6750, "activeOrderCount": 4 }
  ],
  "completedThisWeek": { "orderCount": 3, "quantity": 1024, "from": "2026-09-21" },
  "completedThisMonth": { "orderCount": 24, "quantity": 19410, "from": "2026-09-01" },
  "onTime": { "onTimeCount": 23, "completedCount": 30, "windowStart": "2026-08-24" },

```

```

"leadTime": { "averageDays": 13.7, "orderCount": 30, "windowStart": "2026-08-24" },
"completionTrend": [
  { "weekStart": "2026-06-29", "weekEnd": "2026-07-05", "orderCount": 3 },
  { "weekStart": "2026-09-21", "weekEnd": "2026-09-27", "orderCount": 3 }
]
}

```

(`workload` always has 10 entries and `completionTrend` 12; the example shows the first and last of each. Figures are illustrative, not the seed's exact values.)

Error codes

Code	Meaning	HTTP status
—	Not signed in	401
—	Missing role (client shows MSG-E021)	403
MSG-E013	Unexpected failure (database, timeout)	500

No 400: there is no input to reject. No 404, 409 or 422: the endpoint reads a snapshot of whatever exists, including nothing (REQ-028's empty state is a 200 with zeros).

2. GET /api/system/health

Field	Value
Description	Report whether the database answers a trivial round trip, for SCR-003's health indicator (003_BD HS-01–HS-05)
Return type	<code>SystemHealthResponse</code>
Created by / date	Claude / 2026-09-22
Last modified by / date	—

Arguments: none; a query string is ignored.

Processing overview: authorize with the same policy as the dashboard, run `SELECT 1` with a 2-second timeout (003_DD-FN §6), and answer **200 in both outcomes**. That the endpoint answered at all is the server's status; the body carries only the database's. Requests to this path never renew the sign-in cookie (DEC-019, 003_DD-FN §7).

Processing flow

Step	Description	Calls
1	Authorize	policy <code>ProductionOrderEditor</code>
2	Ping the database	<code>SystemHealthService.CheckAsync</code> → 003_DD-FN §6
3	200 with <code>SystemHealthResponse</code> and <code>Cache-Control: no-store</code> ; the cookie is not reissued	—

Request fields: not applicable.

Response fields — `SystemHealthResponse`

No	Name	Variable name	Type	Length	Required	Repeats (array)	Value mapping	Example	Description	Notes
1	Database	<code>database</code>	string	—	yes	no	<code>ok</code> <code>unavailable</code>	<code>ok</code>	<code>unavailable</code> when the ping failed for any reason or took longer than 2 s	The only component the server reports on (DEC-020)
2	Checked at	<code>checkedAt</code>	string (date-time)	—	yes	no	UTC	<code>2026-09-22T05:05:30Z</code>	When the server ran the check	Shown in plant time (003_BD M-20)

Nothing else is returned: no version, host, connection string, latency or exception text, in either outcome.

```
{ "database": "unavailable", "checkedAt": "2026-09-22T05:05:30Z" }
```

Error codes

Code	Meaning	HTTP status
—	Not signed in (including a session that expired because polling did not renew it)	401
—	Missing role	403
<code>MSG-E013</code>	Unexpected failure outside the ping itself	500 — the browser shows 「サーバー：接続不可」 (Server: Unreachable)

A failing ping is a 200 with `database: "unavailable"`, not an error: the server answered, and reporting that is the endpoint's job.

Unresolved decisions

None. The contract rests on WI-004 DEC-006 (read-only), DEC-008/DEC-009 (windows, buckets), DEC-011 (one endpoint) and DEC-015 (snapshot), and on WI-002 DEC-011/DEC-017 (plant clock), DEC-020 (CSRF) and DEC-023 (RFC 9457). The reuse of the `ProductionOrderEditor` policy is the limitation 002_DD-API already records against WI-001 DEC-015.